

TP8 Java Yassine Hamrouni

Exercice 1 - upperCaseAll

1- Rappeler comment on met une chaîne de caractères en majuscules, indépendamment de la langue dans laquelle l'OS est configuré.

Avec a la methode `toUpperCase(Locale.ENGLISH)`

`Locale.ENGLISH` garantit que la conversion en majuscules suit les règles de la langue anglaise.

2- On va utiliser la méthode `List.replaceAll` pour mettre toutes les chaînes de caractères en majuscules. Quelle est l'interface fonctionnelle utilisée par la méthode `replaceAll` ?

La méthode `List.replaceAll` en Java utilise l'interface fonctionnelle `UnaryOperator`.

```
List<String> list = Arrays.asList("Bonjour", "monde");  
list.replaceAll(s -> s.toUpperCase(Locale.ROOT));
```

3- À quel type de fonction cela correspond-il ? Autrement dit, que prend la fonction en argument et que renvoie-t-elle ?

La fonction utilisée par `UnaryOperator` dans le contexte de la méthode `List.replaceAll` est un type de fonction qui prend un argument de type `T` et renvoie un résultat du même type `T`. En d'autres termes, elle a la signature de fonction suivante :

Entrée : `T` (le type de l'élément de la liste)

Sortie : `T` (le type de l'élément après application de l'opérateur)

4- Sachant que l'on appelle `replaceAll` avec une liste de `String`, quel est le type des paramètres de la lambda et quel est son type de retour ?

le type de paramètre de la lambda est `String` et son type de retour est également `String`.

5- Écrire le code de la méthode `upperCaseAll`

```
public class Lambdas {  
    public static void upperCaseAll(List<String> list) {  
        list.replaceAll(s -> s.toUpperCase(Locale.ENGLISH));  
    }  
}
```

Exercice 2 - upperCaseAll

1 - Quelle est le type du paramètre de la méthode occurrences ? Quelle est le type de retour de la méthode occurrences ?

Type du paramètre de la méthode occurrences :

Le paramètre doit être une liste de chaînes de caractères. En Java, cela se traduit généralement par List. Ce type accepte une collection de chaînes de caractères dont vous voulez compter les occurrences.

Type de retour de la méthode occurrences :

Le type de retour doit être une carte (map) qui associe chaque chaîne de caractères unique à son nombre d'occurrences dans la liste. En Java, cela peut être représenté par Map<String, Integer>, où la clé est la chaîne de caractères (String) et la valeur est le nombre d'occurrences (Integer).

2 - Quelle est l'implantation que l'on doit choisir ici ?

Pour implémenter la méthode occurrences qui calcule le nombre d'occurrences de chaque chaîne dans une liste, l'utilisation d'une HashMap est une solution efficace en termes de complexité temporelle.

2 - Quelle est l'implantation que l'on doit choisir ici ?

La méthode forEach de l'interface Iterable en Java prend en paramètre une implémentation de l'interface fonctionnelle Consumer.

Interface Fonctionnelle Consumer

3 - On veut parcourir la liste avec la méthode forEach. Quelle interface fonctionnelle prend-elle en paramètre ? Quel est le type fonction correspondant ? Ici, quels sont les types des paramètres / de retour de la lambda que vous allez utiliser ?

Interface Fonctionnelle : Consumer

Type Fonctionnel : Cette interface fonctionnelle représente une opération ##### qui accepte un seul argument d'entrée et ne retourne aucun résultat.

Dans le contexte de l'utilisation de `forEach` pour parcourir une liste de chaînes de caractères et calculer les occurrences, voici comment ces types s'appliquent :

Type de l'Argument de la Lambda : `String` (chaque élément de la liste est une chaîne de caractères).

Type de Retour de la Lambda : `void` (la lambda exécutera une action sans renvoyer de valeur).

```
public static Map<String, Integer> occurrences(List<String> strings) {  
    var occurrences = new HashMap<String, Integer>();  
    strings.forEach(string -> occurrences.put(string, occurrences.getOrDefault(string, 0) + 1);  
    return occurrences;  
}
```

4 - Pour compter le nombre d'occurrences, on va utiliser la méthode `merge` de la structure de données que vous avez choisi de renvoyer. Quelle interface fonctionnelle prend-elle en paramètre ? Quel est le type fonction correspondant ? Ici, quels sont les types des paramètres / de retour de la lambda que vous allez utiliser ?

Interface Fonctionnelle utilisée par `merge`

Interface Fonctionnelle : `BiFunction<T, U, R>`

Type Fonctionnel : Cette interface fonctionnelle représente une fonction qui accepte deux arguments et produit un résultat. Elle est utilisée pour définir la logique de combinaison des valeurs existantes et nouvelles pour une clé donnée dans une `Map`.

Dans le cas de l'utilisation de la méthode `merge` pour compter les occurrences de chaînes dans une liste :

Types des Paramètres de la Lambda :

T (premier paramètre) : `Integer` - la valeur actuelle associée à la clé (l'ancienne valeur d'occurrence).

U (deuxième paramètre) : `Integer` - la nouvelle valeur à combiner (dans notre cas, cela sera typiquement 1, le compteur à ajouter).

Type de Retour de la Lambda : `Integer` - la nouvelle valeur mise à jour qui sera stockée dans la `map`.

5- Écrire le code de occurrences, toujours dans la classe Lambdas.

```
package info.esiee.tp8;

import java.util.HashMap;
import java.util.List;
import java.util.Locale;
import java.util.Map;

public class Lambdas {
    public static void upperCaseAll(List<String> list) {
        list.replaceAll(s -> s.toUpperCase(Locale.ENGLISH));
    }

    // public static Map<String, Integer> occurrences(List<String> strings) {
    //     Map<String, Integer> occurrences = new HashMap<>();
    //     strings.forEach(string -> occurrences.put(string, occurrences.getOrDefault(string, 0)
    //     return occurrences;
    // }

    public static Map<String, Integer> occurrences(List<String> strings) {
        Map<String, Integer> occurrences = new HashMap<>();
        strings.forEach(string -> occurrences.merge(string, 1, (oldValue, value) -> oldValue + v
        return occurrences;
    }
}
```

6- On peut noter qu'il existe une méthode statique sum dans la classe java.lang.Integer qui fait la somme de deux valeurs, on peut donc l'utiliser sous forme de method reference à la place de la lambda, lors de l'appel à merge. Modifier le code pour l'utiliser (garder la précédente version en commentaires).

```
public static Map<String, Integer> occurrences(List<String> strings) {
    Objects.requireNonNull(strings);
    Map<String, Integer> occurrences = new HashMap<>();
    strings.forEach(string -> occurrences.merge(string, 1, Integer::sum));
    return occurrences;
}
```

Exercice 3 - groupBy

1 - Quel est le type de paramètre de actorGroupByFirstName ? Quel est le type de retour de actorGroupByFirstName ?

Le paramètre est une liste< et le retour est un Map<String, Actor>

2 - Rappeler comment marche la méthode `Map.computeIfAbsent`. Son second paramètre est une interface fonctionnelle, à quel type de fonction correspond-elle ? Expliquer à quoi correspondent le premier paramètre et le second paramètre de `Map.computeIfAbsent`, puis comment on peut l'utiliser pour grouper les acteurs selon leur prénom.

La méthode `Map.computeIfAbsent` tente de trouver une valeur associée à une clé spécifique. Si cette clé n'existe pas dans la map, elle utilise la fonction fournie (second paramètre) pour créer une nouvelle valeur, la place dans la map et la retourne.

Type de fonction : `Function<? super K, ? extends V>`

Premier paramètre : La clé à rechercher dans la map.

Second paramètre : Une fonction qui fournit une valeur par défaut si la clé n'est pas présente.

3- Dans notre cas, quel doit être le type de la lambda passée en second paramètre de `computeIfAbsent` ?

Type de la lambda : `Function<String, List>`

4- Écrire la méthode `actorGroupByFirstName()`

```
import java.util.*;
import java.util.stream.Collectors;

public class ActorGrouping {

    public record Actor(String firstName, String lastName) {
        public Actor {
            Objects.requireNonNull(firstName);
            Objects.requireNonNull(lastName);
        }
    }

    public static Map<String, List<Actor>> actorGroupByFirstName(List<Actor> actors) {
        return actors.stream()
            .collect(Collectors.groupingBy(Actor::firstName));
    }
}
```

5- Si on veut maintenant grouper les acteurs par rapport à leur nom (lastName) au lieu du prénom, on va écrire à peu près le même code. On veut généraliser le code en écrivant une méthode actorGroupBy qui prend en paramètre une liste d'acteurs ainsi qu'une fonction qui, étant donné un acteur, renvoie la valeur par laquelle il va être groupé.

```
import java.util.List;
import java.util.Map;
import java.util.function.Function;
import java.util.stream.Collectors;

public class Actor {

    // Méthode pour grouper les acteurs par prénom
    public static Map<String, List<Actor>> actorGroupByFirstName(List<Actor> actors) {
        return actors.stream()
            .collect(Collectors.groupingBy(Actor::firstName));
    }

    // Méthode générique pour grouper les acteurs par une fonction donnée
    public static Map<String, List<Actor>> actorGroupBy(List<Actor> actors, Function<Actor,
        return actors.stream()
            .collect(Collectors.groupingBy(groupByFunction));
    }
}
```

Quelle doit être le type fonction du second paramètre de actorGroupBy ?

Type fonction : Function<Actor, String>

Quelle est l'interface fonctionnelle correspondante ?

Interface fonctionnelle : Function

Quelle doit être le type du second paramètre de actorGroupBy ?

Type du second paramètre : Function<Actor, String>

Écrire la méthode actorGroupBy.

```
import java.util.List;
import java.util.Map;
import java.util.function.Function;
import java.util.stream.Collectors;
```

```

public class Actor {

    // Méthode pour grouper les acteurs par prénom
    public static Map<String, List<Actor>> actorGroupByFirstName(List<Actor> actors) {
        return actors.stream()
            .collect(Collectors.groupingBy(Actor::firstName));
    }

    // Méthode générique pour grouper les acteurs par une fonction donnée
    public static Map<String, List<Actor>> actorGroupBy(List<Actor> actors, Function<Actor,
        return actors.stream()
            .collect(Collectors.groupingBy(groupByFunction));
    }
}

```